MASTERARBEIT

PRÜFUNGSNR.: XXXXXX

PRÜFER: PROF. DR.-ING. JENS B. SCHMITT

EXPOSÉ

22. Januar 2026

Mihir Mahesh Ambli

Mtrk.nr.: 426197

Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau (RPTU)

xed48sit@rptu.de

Contents

1 Concept **1**

2 Literature Review **2**

3 Research Question **3**

4 Approach **3**

5 Methods **4**

6 Outline **4**

7 Supervisor **5**

8 Citations **5**

1 Concept

Background Traditional web security relies on perimeter defense (WAFs, Firewalls) and signature-based detection. These methods are increasingly failing against sophisticated, human-operated attacks and automated botnets that mimic legitimate user behavior. Active Defense, specifically cyber deception, offers a paradigm shift by transforming the web infrastructure from a passive target into a hostile environment for attackers.

However, current deception strategies suffer from two major limitations:

- Static Nature: Honeypots and Honeytokens are often static assets (e.g., a permanent fake file). Once identified by the attacker community, they are easily fingerprinted and avoided.
- Deployment Complexity: Integrating deception requires significant manual configuration, often deterring administrators from adopting these measures.

This thesis proposes a dynamic, server-side deception framework, utilizing the ngx_http_lua_module (OpenResty) and NodeJS. Instead of static decoys, this system will inject honeytokens (e.g., fake credentials in HTML comments) into HTTP responses in real-time.

Lua, specifically via the ngx_http_lua_module (OpenResty), is a primary choice for this framework due to its high-performance Just-In-Time (JIT) compilation and non-blocking asynchronous architecture. These characteristics allow for deep integration into the Nginx request-processing lifecycle, enabling real-time modification of response streams (e.g., via body_filter) with minimal latency. This ensures that dynamic honey tokens can be served to all sessions without introducing acceptable overhead or impacting the performance of legitimate application traffic. Unlike traditional honeypots that require separate infrastructure, this framework utilizes the server's native event-driven architecture to serve lures that are invisible to legitimate users but irresistible to adversaries.

Relevance The scientific contribution of this work is the development of a injection engine that resides directly within the web server's request processing lifecycle and works in shifting the detection paradigm from probabilistic triggers to deterministic, post-interaction confirmation. This solves key operational hurdles:

1. **Operational Efficiency:** It minimizes the risk of ”pollution” (legitimate users seeing decoys) by targeting only high-risk sessions.

2. **Usability:** It abstracts complex deception logic into a simple configuration model for administrators, automating the deployment of honeytokens for attackers.

3. **Omnipresent Deployment:** By serving honey tokens to all sessions, the system removes the need for pre-emptive attacker detection, ensuring that any malicious actor performing reconnaissance is immediately presented with a trap.

2 Literature Review

Current research into Web Server Deception (WSD) focuses largely on standalone honeypots (e.g., Glastopf, Snare) or heavy-weight deception platforms (e.g., T-Pot). State-of-the-art research in web deception has moved from standalone honeypots toward production-integrated lures. It highlights the necessity for honey tokens to be indistinguishable from real data to avoid detection by advanced adversaries.

- **Automated Token Generation (HoneyGen):** Bercovitch et al. (2011) introduced HoneyGen, a novel method for generating honey tokens by mining rules from production databases to create synthetic data that mimics the statistical and structural properties of real information. (2)

- **Tactical Lure Evaluation (The PARCS Model):** Developed by Tournyol-Du-Clos and Faedda (2024), the PARCS model provides a framework for evaluating lures based on Pertinence, Attractiveness, Risk, Credibility, and Scalability. (1)

- **Stealth and Fingerprinting:** Msaad et al. (2022) in ”Honeysweeper” demonstrate that advanced attackers can often ”fingerprint” open-source honey tokens without triggering an alert. (3)

- **Generative AI in Deception:** Reti et al. (2024) explore using Large Language Models to automatically generate convincing deceptive content, such as robots.txt lures and ”honeywords,” to overcome the scalability challenges of manual token creation. (4)

3 Research Question

Main Research Question RQ1: How can web server extensibility modules (Lua/NJS) be utilized to implement a single phase active defense mechanism that serves hidden honey tokens to all users and flags malicious actors solely upon token interaction?

Sub-questions:

- **RQ1.1 Web Artifact Deception Matrix (WADM):** Construct a matrix that maps honey token types (e.g., hidden comments, fake cookies, decoy .env files) against their interaction fidelity and administrative risk.
- **Matrix Criteria:** What criteria (e.g., the PARCS model: Pertinence, Attractiveness, Risk, Credibility, and Scalability) must be satisfied to implement a high-fidelity deception matrix?
- **Injection Mechanics:** How can Lua/NJS inject these tokens into the HTML response stream for all users without introducing unacceptable latency or breaking application functionality?

4 Approach

Single-Phase Unified Middleware Logic: The middleware executes a unified loop within the server's event cycle:

- **Omnipresent Injection (Response Phase):** For every outgoing HTTP response, the script utilizes body_filter_by_lua to dynamically inject the honey tokens defined in the WADM (e.g., fake database credentials in an HTML comment).
- **Deterministic Monitoring (Request Phase):** Simultaneously, the middleware monitors all incoming requests in the access phase. It specifically watches for the utilization of the unique fake credentials or access to the decoy URLs defined in the matrix.
- **Immediate Flagging:** Because these tokens are served to everyone but have no functional use for legitimate users, any interaction results in the user being immediately and deterministicly flagged as an "Attacker". This ensures the defense is always "on" without requiring pre-identification of threats.

5 Methods

- **Prototyping:** Development of an Nginx plugin (Lua/NJS) that allows an administrator to input decoy data via a simple config file.
- **Simulation:** The PoC will be tested against a standard LAMP stack application. ”Attacker” behavior will be simulated using tools like OWASP ZAP and Nikto to measure detection rates.
- **Benchmarking:** The performance impact (Time to First Byte - TTFB) of the injection logic will be measured to ensure the solution is viable.

6 Outline

1. Introduction

   1.1 The Asymmetry of Cyber Defense

   2.2 Problem Statement: The Static Nature of Deception

   3.3 Research Goals and Scope

2. Theoretical Background

   2.1 Active Defense: Honeypots and Honey Tokens

   2.2 Web Server Architecture (Nginx Event Loop)

   2.3 Attacker Behavior Profiling (OWASP Indicators)

3. Related Work

   3.1 Comparison of Existing Honeypot Architectures

   3.2 Server-Side Scripting for Security

4. Experiment

   1.1 Overview

   2.2 Data

   3.3 Methods

4.4 Results

5. Discussion

5.1 Limitations

5.2 Future Work

6. Conclusion & perspective

7 Supervisor

Daniel Reti

8 Citations

References

Deceptive Security: the solution for effective detection in the cloud. ://www.riskinsight-wavestone.com/en/2024/01/deceptive-security-the-solution-for-effective-detection-in-the-cloud-your-luring-strategy/.

Maya Bercovitch, Meir Renford, Lior Hasson, Asaf Shabtai, Lior Rokach, and Yuval Elovici. Honeygen: An automated honeytokens generator. In *Proceedings of 2011 IEEE International Conference on Intelligence and Security Informatics*, pages 131–136, 2011.

Mohamed Msaad, Shreyas Srinivasa, Mikkel M. Andersen, David H. Audran, Charity U. Orji, and Emmanouil Vasilomanolakis. Honeysweeper: Towards stealthy honeytoken fingerprinting techniques. In Hans P. Reiser and Marcel Kyas, editors, *Secure IT Systems*, pages 101–119, Cham, 2022. Springer International Publishing.

Daniel Reti, Norman Becker, Tillmann Angeli, Anasuya Chattopadhyay, Daniel Schneider, Sebastian Vollmer, and Hans D. Schotten. Act as a honeytoken generator! an investigation into honeytoken generation with large language models. In *Proceedings of the 11th ACM Workshop on Adaptive and Autonomous Cyber Defense*, AACD '24, page 1–12, New York, NY, USA, 2024. Association for Computing Machinery.